MODULE 4

Reinforcement Learning (RL)

1.1 What is Reinforcement Learning?

Reinforcement Learning (RL) is a type of machine learning in which an agent learns how to make decisions by interacting with an environment and receiving rewards or penalties.

The goal is to learn a strategy that maximizes the total reward over time.

Example: A robot learns to navigate a room. It receives a positive reward for reaching its destination and a penalty for hitting obstacles.



1.2 Important terms

- **Agent**: The learner that selects actions.
- **Environment**: The external world with which the agent interacts.
- **State (s):** The current situation of the agent.
- **Action (a):** A choice made by the agent.
- **Reward (r):** Feedback indicating how good or bad an outcome is.
- **Policy (π):** A strategy for selecting actions in different states.
- **Return**: The total accumulated reward.
- **Value function**: The expected long-term reward from a state or state-action pair.

1.3 Markov Decision Process (MDP)

Reinforcement learning is commonly modelled using a Markov Decision Process (MDP).

An MDP consists of:

1. S: Set of states.
2. A: Set of actions.

3. $T(s, a, s')$: Probability of reaching state $s'$from state $s$after action a.
4. $R(s, a, s')$: Reward received during a transition.
5. γ: Discount factor, between 0 and 1, controlling the importance of future rewards.

The Markov property means that the next state depends on the current state and action, rather than the entire history, assuming the state captures all relevant information.

1.4 Discounted return

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

- If $\gamma$is close to 0, immediate rewards are emphasized.
- If $\gamma$is close to 1, future rewards are given more importance.

1.5 Types of reinforcement learning

Type	**Main idea**
Passive RL	The policy is fixed; the agent learns how good it is.
Active RL	The agent chooses actions and learns which are best.
Model-based RL	Uses or learns a model of the environment.
Model-free RL	Learns values or policies without explicitly learning the environment's transition model.

Key point: The difference between passive and active reinforcement learning is whether the agent follows a fixed policy or actively chooses actions to improve its behavior.

2. Passive Reinforcement Learning

2.1 Definition

In passive reinforcement learning, the agent follows a given, fixed policy and learns the utility or value of the states it visits.

The agent does not decide which policy is best. It evaluates the policy it has been given.

Example: A delivery robot follows a predetermined route and learns the expected reward associated with each location.

2.2 Main approaches

A. Direct utility estimation

- The agent observes the rewards obtained during an episode.
- It calculates the total return for each visited state.
- It averages the observed returns to estimate the utility of each state.
- Direct utility estimation succeeds in reducing the reinforcement learning problem to an inductive learning problem, about which much is known.
- The utility of each state equals its own reward plus the expected utility of its successor states. That is, the utility values obey the Bellman equations for a fixed policy.

For state s:

$$U^{\pi}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi(s)) U^{\pi}(s') .$$

where γ is discount factor.

Advantages: Simple and easy to implement.

Limitations: Requires sufficient experience and can have high variance.

B. Adaptive dynamic programming (ADP)

ADP learns a model of the environment and uses it to calculate state utilities.

The agent estimates:

- Transition probabilities.
- Expected rewards.

Advantages: Can make effective use of a learned model.

Limitations: Requires model estimation and may be computationally expensive for large state spaces.

C. Temporal-difference (TD) learning

- TD learning updates a state's estimated utility using the reward received and the estimated utility of the next state.
- When a transition occurs from state s to states' we apply the following update to $U^{\pi}(s)$:Type equation here.

$$U^{\pi}(s) \leftarrow U^{\pi}(s) + \alpha(R(s) + \gamma U^{\pi}(s') - U^{\pi}(s)) .$$

where:

- α= learning rate.
- R= observed reward.
- s'= next state.
- γ= discount factor.

The expression

$$R(s) + \gamma U(s') - U(s)$$

is the difference in utilities between successive states, it is often called the **temporal-difference**, or TD equation.

Advantages: Learns directly from experience without requiring a complete model.

Limitations: Estimates may be inaccurate when experience is limited.

2.3 Comparison of passive RL methods

Method	**Learns from**	**Needs an environment model?**
Direct utility estimation	Complete observed returns	No
ADP	Learned transition and reward model	Yes, or an estimated model
TD learning	Immediate reward and next-state estimate	No

3. Active Reinforcement Learning

3.1 Definition

In active reinforcement learning, the agent chooses actions, observes their results, and learns which actions produce the highest long-term reward.

Unlike passive RL, the agent must learn both:

- How good the current states and actions are.
- Which actions it should choose to improve its performance.

Example: A robot explores different routes to discover which route reaches its destination safely and quickly.

3.2 Main concepts

1. **Exploration**: Trying actions to gain new information about the environment.

2. **Exploitation**: Choosing the action currently believed to be the best.

A good RL agent must balance exploration and exploitation.

3.3 Q-learning (Off Policy RL)

Q-learning is a widely used model-free, off-policy reinforcement learning algorithm.

It learns the action-value function $Q(s, a)$, which represents the expected discounted return from taking action a in state $s$and then following the best policy.

The update equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R(s) + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where:

- $Q(s, a)$: Current estimate of action value.
- α: Learning rate.
- r: Immediate reward.
- γ: Discount factor.
- s': Next state.
- $\max_{a'} Q(s', a')$: Highest estimated value of the next state's available actions.

How Q-learning works:

1. Initialize the Q-values.
2. Observe the current state s.
3. Choose an action a, allowing exploration.
4. Perform the action and observe reward $r$and next state s'.
5. Update $Q(s, a)$using the Q-learning equation.
6. Repeat until the values or learned behaviour become sufficiently stable.

The learned policy usually chooses,

$$\pi(s) = \arg\max_{a} Q(s, a)$$

3.4 SARSA (On-Policy Algorithm):

SARSA stands for State-Action-Reward-State-Action, representing the tuple *(s, a, r, s', a')*. Unlike Q-learning, SARSA is an on-policy method. Its update rule relies on the actual next action *a'* selected by the agent's current behavioural policy:

$Q(s, a) \leftarrow Q(s, a) + \alpha [R(s, a) + \gamma Q(s', a') - Q(s, a)]$

SARSA takes exploration risks into account during training, making it safer in environments where exploratory actions carry high penalties or costs.

3.5 Passive RL vs Active RL

Passive RL	Active RL
Follows a fixed policy	Selects actions to improve its behaviour
Evaluates a policy	Learns to select a good policy
Focuses on state utility under that policy	Often learns action values
No policy exploration is required	Exploration is generally important

4. Generalization in Reinforcement Learning

4.1 Definition

Generalization is the ability of a reinforcement learning agent to apply what it has learned in familiar states to new or previously unseen states.

In large problems, it is usually impossible to store a separate value for every state-action pair. Generalization helps the agent learn from limited experience.

Example: A robot trained to navigate several rooms can use the knowledge it has acquired to navigate a new room with a similar layout.

4.2 Why is generalization needed?

- Real-world environments can have enormous state spaces.
- Many states may never be visited during training.
- Learning a separate value for every state is expensive.
- Similar states may require similar actions.

4.3 Methods of generalization

A. Feature-based representation

Represent each state using useful features rather than treating every state as completely different.

For example, a self-driving robot might use:

- Distance to an obstacle.
- Current speed.
- Distance to the destination.
- Direction of movement.

A value function can then be approximated as

$$\widehat{U}(s) = w^T x(s)$$

where $x(s)$is the feature vector and w is a vector of learned weights.

B. Function approximation

A parameterized function approximates state values or action values.

Examples include:

- Linear models.
- Decision trees.
- Neural networks.

For example,

$$\hat{Q}(s, a, w) \approx Q(s, a)$$

A neural network can learn complex relationships between states and actions.

C. State abstraction

Similar states are grouped together when their relevant behaviour can be represented by a common abstract state.

For example, different positions on a map might be grouped into the same region if they have similar rewards and action outcomes.

4.4 Advantages and limitations

Advantages	Limitations
Reduces memory requirements	Poor features can produce poor decisions
Helps with unseen states	Incorrect generalization may be misleading
Supports large problems	Function approximation may be difficult to train
Can improve learning efficiency	Training data may not cover important situations

5. Policy Search

Policy search is an approach to reinforcement learning that directly searches for a policy that maximizes expected cumulative reward.

Instead of first learning an accurate value function for every state, policy search adjusts the policy's parameters to improve performance.

A policy may be represented as

$$\pi_\theta(s, a)$$

where θrepresents the policy parameters.

5.2 Types of policy search

1. Direct policy search

- Define a parameterized policy.
- Evaluate its performance through interaction or simulation.
- Adjust parameters to improve the total reward.

2. Gradient-based policy search

Uses policy gradients to improve the policy parameters:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

Here, $\nabla_\theta J(\theta)$indicates the direction in parameter space that locally increases expected reward.

3. Stochastic policy search

The policy selects actions according to probabilities rather than always choosing the same action in a given state. This can help exploration and handle uncertainty.

5.3 General procedure

1. Choose a policy representation with parameters θ.
2. Execute the policy in the environment or a simulator.
3. Measure the cumulative reward.
4. Adjust the parameters to improve expected performance.
5. Repeat until a satisfactory policy is found.

5.4 Advantages

- Directly optimizes the behaviour of the agent.
- Can naturally handle continuous action spaces.
- Can represent complex policies, including neural-network policies.
- Does not always require explicit value-function estimation.

5.5 Limitations

- Evaluating a policy may require many trials.
- Performance estimates can be noisy.
- Optimization may find a local optimum rather than the global optimum.
- Training can be computationally expensive.

5.6 Policy search versus value-based learning

Policy search	**Value-based learning**
Directly optimizes the policy	Learns values of states or actions
Useful for continuous actions and complex policies	Convenient for discrete action selection

Often uses policy optimization or gradients	Often selects the action with the highest estimated value
Can require many policy evaluations	Can also be expensive, depending on the problem

6. Apprenticeship Learning and Inverse Reinforcement Learning

6.1 Apprenticeship learning

Apprenticeship learning is a method in which an agent learns to perform a task by observing demonstrations from an expert.

Instead of learning entirely through trial and error, the agent uses expert behavior as guidance.

Example: A robot observes a human moving objects and learns to perform a similar task.

Typical steps:

1. Collect expert demonstrations.
2. Identify patterns in the expert's behavior.
3. Learn a policy or infer the objective behind the behavior.
4. Evaluate the learned behavior.
5. Improve it using additional demonstrations or interaction.

6.2 Inverse Reinforcement Learning (IRL)

Inverse Reinforcement Learning attempts to infer the reward function that explains an expert's behavior.

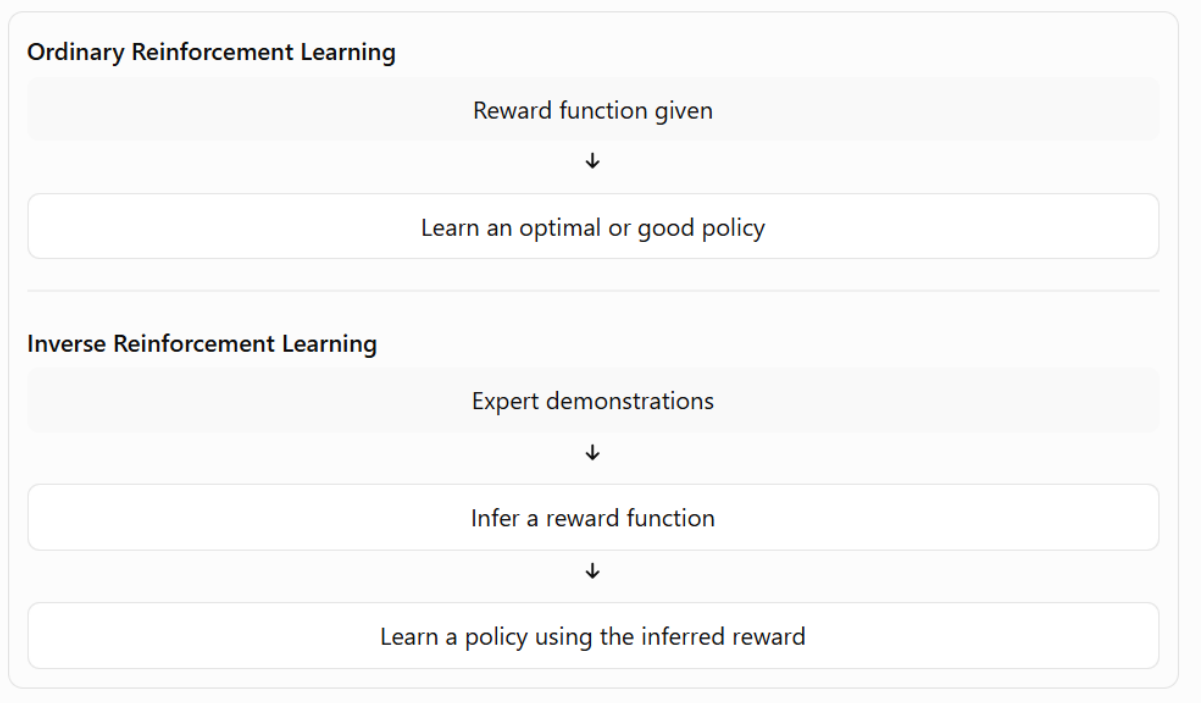
In ordinary reinforcement learning, the reward function is given or specified. In IRL, the agent observes expert behavior and tries to determine what the expert is trying to achieve.

For example:

- Expert behavior: A driver slows down near pedestrians.
- Possible inferred reward: Avoid collisions and maintain safety.
- Learned policy: Drive cautiously when pedestrians are nearby.

The inferred reward is not necessarily unique; several reward functions may explain the same behavior.

6.3



6.4 Apprenticeship learning versus IRL

Apprenticeship learning	**Inverse reinforcement learning**
Broad approach to learning from expert demonstrations	Specific approach that infers the reward function
May directly imitate expert actions	Attempts to discover the expert's underlying objective
Can use imitation learning or IRL	Uses expert behavior to estimate rewards
Focuses on acquiring expert-like behavior	Focuses on identifying the reward that explains behavior

6.5 Advantages and limitations

Advantages

- Reduces the need for random exploration.
- Can use human knowledge to guide learning.
- Useful when manually defining a reward function is difficult.
- Can support complex tasks where the objective is easier to demonstrate than to describe.

Limitations

- Expert demonstrations may be incomplete or imperfect.
- Demonstrations can be expensive to collect.
- The intended reward may be ambiguous.
- Learned behavior may not generalize to situations absent from the demonstrations.

7. Applications of Reinforcement Learning

Reinforcement learning is useful when actions influence future outcomes and the agent can improve through feedback.

1**. Robotics and automation**

Robots learn movement, object manipulation, path planning, and control strategies.

Example: A robotic arm learns how to pick and place objects accurately.

2. **Autonomous vehicles**

RL can help optimize simulated driving policies, lane changes, and traffic responses.

Example: Learning a policy that balances smooth driving and safe following distances.

3. Games

Agents learn strategies through rewards, simulations, and repeated play.

Example: Learning strong strategies in chess or other games.

4. Power systems and energy management

RL can optimize energy storage, building energy use, and resource allocation.

Example: Scheduling battery charging to reduce electricity costs while meeting operational constraints.

5. Computer networks

RL can support routing, congestion management, and allocation of network resources.

Example: Selecting routing policies that reduce delay and congestion.

6. Healthcare

RL research explores sequential treatment decisions and resource scheduling.

Example: Optimizing treatment strategies under carefully validated clinical constraints.

Benefits of reinforcement learning

- Learns from interaction and feedback.
- Optimizes long-term outcomes.
- Can adapt to changing environments.
- Can handle sequential decision-making problems.
- Can discover strategies that are difficult to program manually.

Challenges in real-world applications

- Requires sufficient and useful training experience.
- Exploration can be costly or unsafe.
- Poorly designed rewards can produce undesirable behavior.
- Results may not transfer from simulation to the real world.
- Safety, reliability, and computational cost must be considered.

For engineering applications, RL is often trained in simulation and evaluated under strict constraints before deployment in physical systems.